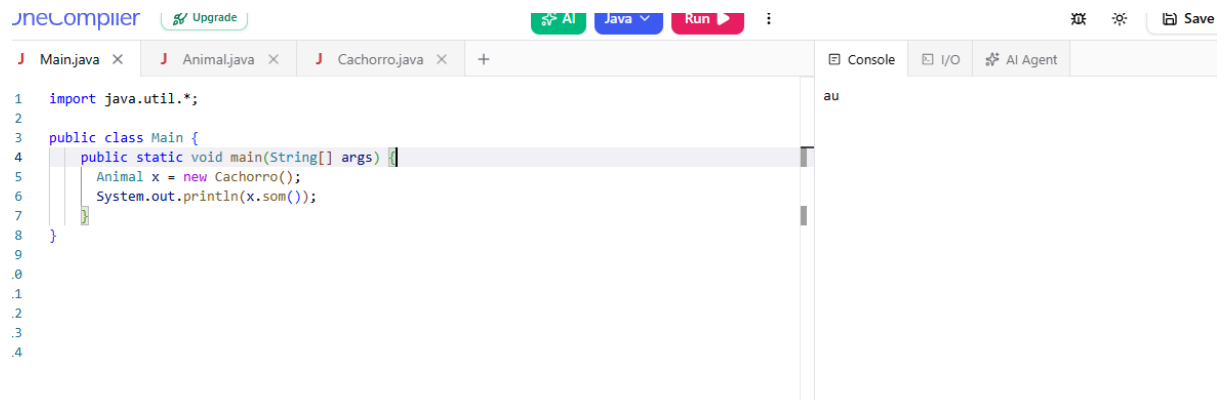


1. Java

a)

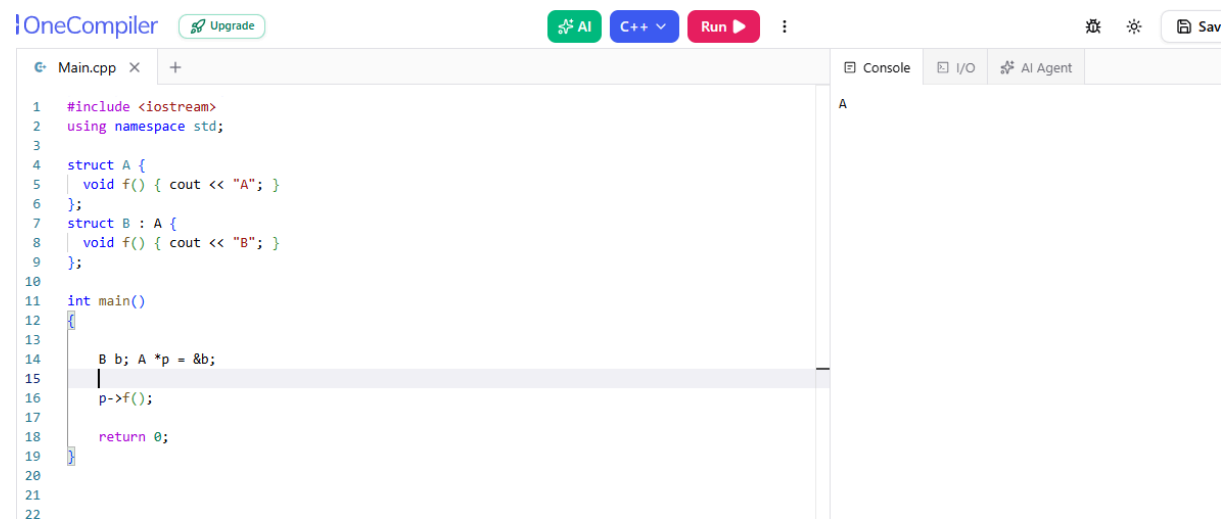


```
1 import java.util.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         Animal x = new Cachorro();
6         System.out.println(x.som());
7     }
8 }
9
10
11
12
13
14
```

b) A decisão tomada foi que, entre o método da classe Animal e o da classe Cachorro, quem foi chamado foi o da classe Cachorro mesmo.

2. C++

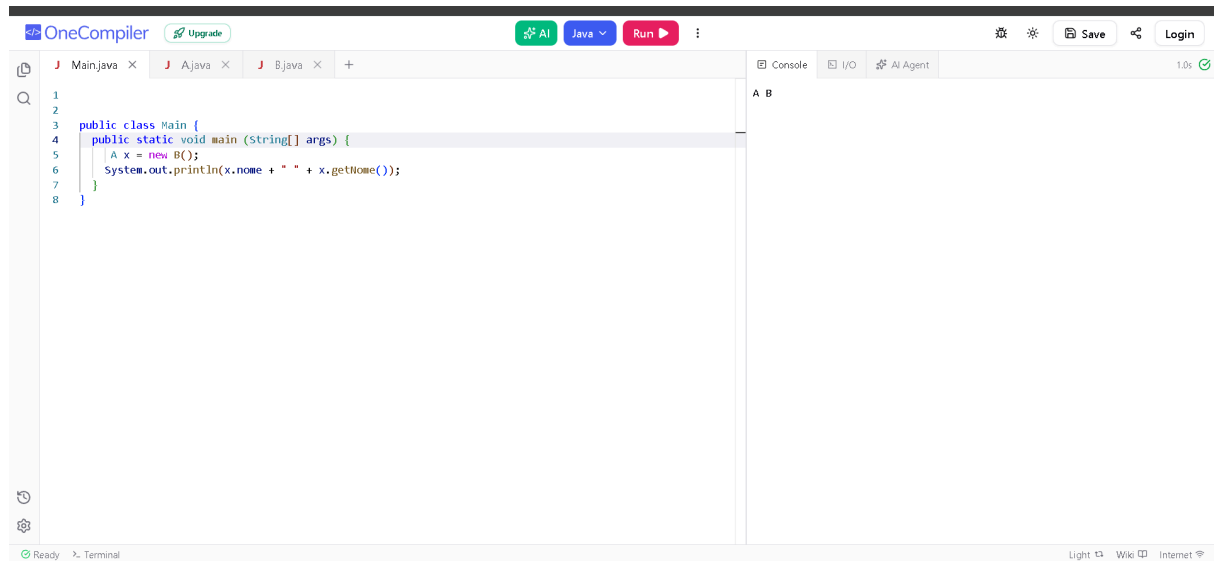
a)



```
1 #include <iostream>
2 using namespace std;
3
4 struct A {
5     void f() { cout << "A"; }
6 };
7 struct B : A {
8     void f() { cout << "B"; }
9 };
10
11 int main()
12 {
13     B b; A *p = &b;
14     p->f();
15     return 0;
16 }
17
18
19
20
21
22
```

b) Mesmo que o ponteiro do tipo A receba o endereço de B, o método chamado ainda é o do A, não o do B.

3. Java



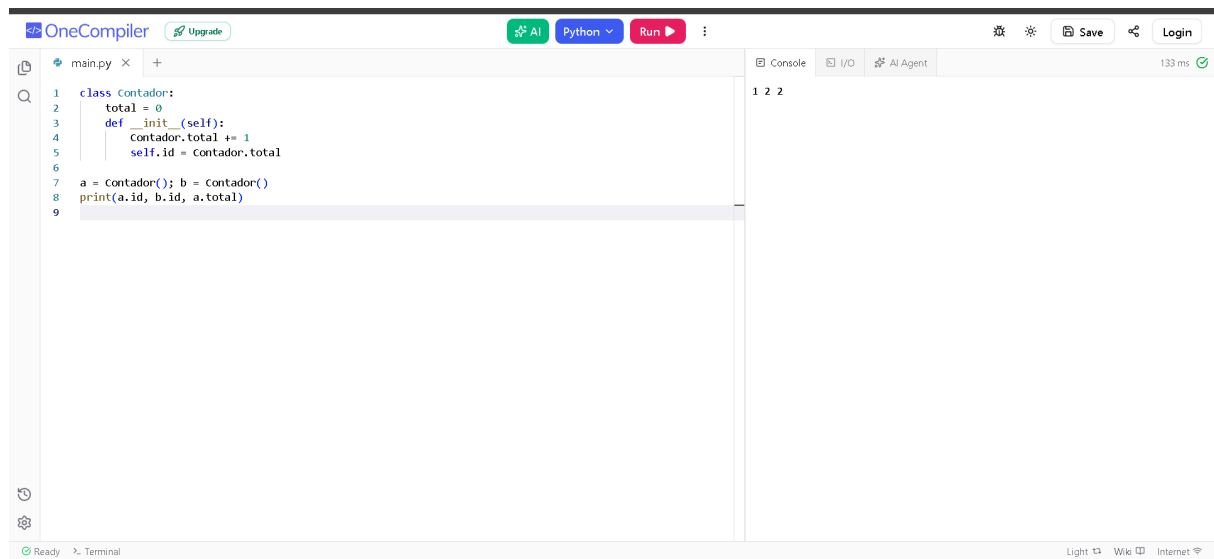
The screenshot shows the OneCompiler Java IDE. The editor has a tab for 'Main.java'. The code is as follows:

```
1  
2  
3  
4 public class Main {  
5     public static void main (String[] args) {  
6         A x = new B();  
7         System.out.println(x.nome + " " + x.getNome());  
8     }  
9 }
```

The right-hand pane shows the output 'A B' and the execution time '1.0s'. The status bar at the bottom indicates 'Ready' and 'Terminal'.

b) CAMPOS não são polimórficos: `x.nome` usa o tipo declarado (A), e o campo `nome` de B apenas **ESCONDE** o de A. Já o MÉTODO `getNome()` é despachado dinamicamente e devolve o nome de B. É um motivo para manter campos privados.

4. Python



The screenshot shows the OneCompiler Python IDE. The editor has a tab for 'main.py'. The code is as follows:

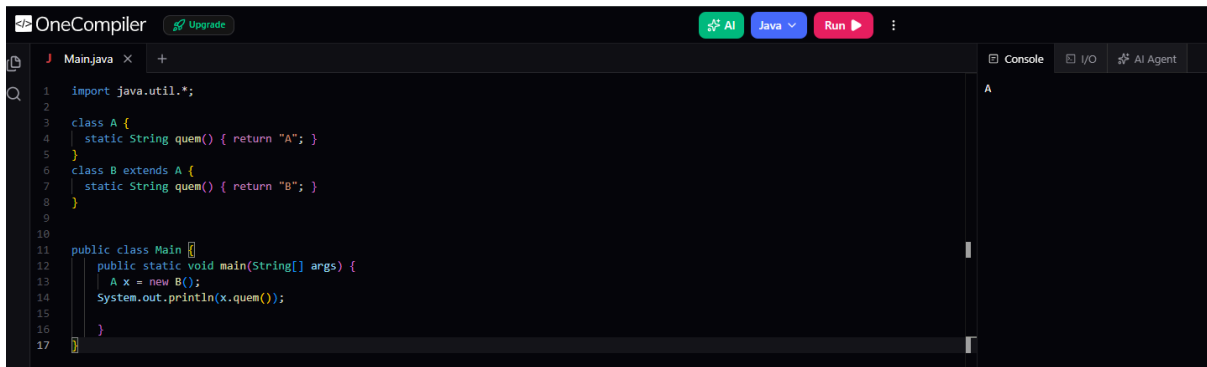
```
1 class Contador:  
2     total = 0  
3     def __init__(self):  
4         Contador.total += 1  
5         self.id = Contador.total  
6  
7 a = Contador(); b = Contador()  
8 print(a.id, b.id, a.total)  
9
```

The right-hand pane shows the output '1 2 2' and the execution time '133 ms'. The status bar at the bottom indicates 'Ready' and 'Terminal'.

b) `total` é uma variável de CLASSE (uma cópia só, compartilhada), e `id` é de INSTÂNCIA (uma por objeto). `a.total` encontra o total da class

5. Java

a) saída



The screenshot shows the OneCompiler IDE with a Java file named 'Main.java'. The code defines two classes, A and B, where B extends A. Both classes have a static method 'quem()' that returns a string. Class A's 'quem()' returns 'A', and Class B's 'quem()' returns 'B'. The 'Main' class has a 'main' method that creates an instance of B and calls its 'quem()' method. The output in the console is 'A'.

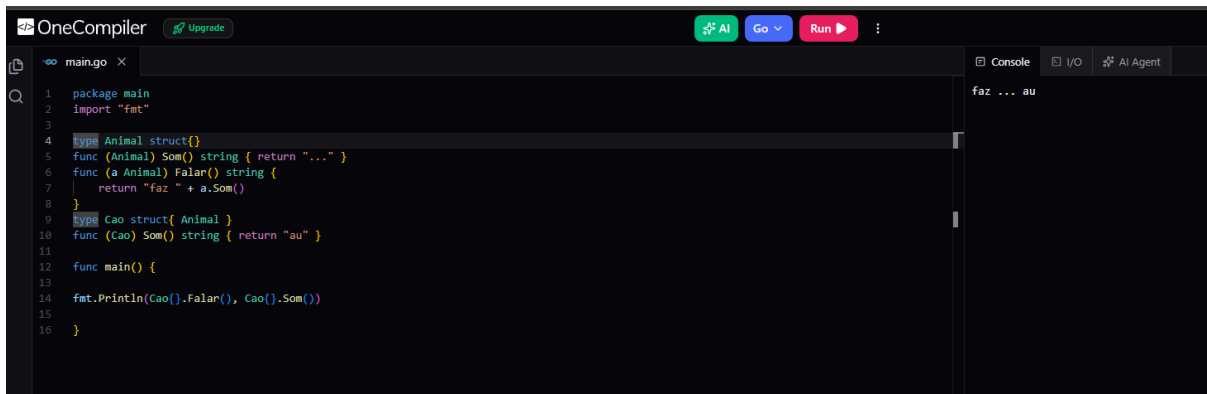
```
1 import java.util.*;
2
3 class A {
4     static String quem() { return "A"; }
5 }
6 class B extends A {
7     static String quem() { return "B"; }
8 }
9
10
11 public class Main {
12     public static void main(String[] args) {
13         A x = new B();
14         System.out.println(x.quem());
15     }
16 }
17
```

Console: A

b) A saída é A por que o java considera que x seria do tipo estatuído A, que é um objeto e B seria o tipo dinâmico, e como a linha que chama x.quem(), o java considera X do tipo A, por tanto a saída é A

6. GO

a) saída



The screenshot shows the OneCompiler IDE with a Go file named 'main.go'. The code defines two structs, 'Animal' and 'Cao', and two functions, 'Som' and 'Falar'. The 'Animal' struct has a 'Som' method that returns '...'. The 'Cao' struct has a 'Som' method that returns 'au'. The 'Falar' function takes an 'Animal' struct and returns a string. The 'main' function calls 'Falar' with a 'Cao' struct. The output in the console is 'faz ... au'.

```
1 package main
2 import "fmt"
3
4 type Animal struct{}
5 func (Animal) Som() string { return "..." }
6 func (a Animal) Falar() string {
7     return "faz " + a.Som()
8 }
9 type Cao struct{ Animal }
10 func (Cao) Som() string { return "au" }
11
12 func main() {
13     fmt.Println(Cao{}.Falar(), Cao{}.Som())
14 }
15
16
```

Console: faz ... au

b) Isso ocorre por que o GO faz uma composição quando é criado a classe do cão, sendo assim cão recebe as propriedades da classe animal, sendo semelhante à herança em java